

AWS IAM (Identity and Access Management) - Service Overview

Introduction

AWS Identity and Access Management (IAM) enables administrators to securely control access to AWS services and resources. IAM allows you to create and manage AWS users and groups, and use permissions to allow or deny their access to AWS resources. IAM is offered at no additional charge.

Core Concepts

- Users: an entity representing a person or application that interacts with AWS. Each user has unique security credentials.
- Groups: a collection of IAM users. Attaching a policy to a group applies it to all users within that group.
- Roles: an identity with specific permissions that can be assumed temporarily by trusted entities, such as EC2 instances, Lambda functions, or federated users. Roles do not have long-term credentials like passwords or access keys.
- Policies: JSON documents that define permissions. A policy specifies what actions are allowed or denied on which resources, and under what conditions.

Policy Structure

An IAM policy consists of one or more statements, each with:

- Effect: "Allow" or "Deny"
- Action: the specific API operations covered, e.g. "s3:GetObject"
- Resource: the ARN(s) the statement applies to
- Condition (optional): additional constraints, such as requiring MFA or restricting by source IP

Principle of Least Privilege

AWS strongly recommends granting only the permissions required to perform a task, and no more. Wildcard resource ("*") and wildcard action permissions should be avoided in production policies. IAM Access Analyzer can help identify resources shared with external entities and generate least-privilege policies based on observed access activity.

IAM Roles for Services

Rather than storing long-lived access keys inside an application, AWS recommends attaching an IAM role to the compute resource (EC2 instance, ECS task, Lambda function). The AWS SDK

automatically retrieves temporary credentials from the instance metadata service or task role, and these credentials rotate automatically.

Multi-Factor Authentication (MFA)

IAM supports MFA using virtual authenticator apps, hardware tokens, or FIDO security keys. MFA should be enabled on the AWS account root user and on all IAM users with console access, particularly those with administrative privileges.

IAM Identity Center (formerly AWS SSO)

For organizations managing multiple AWS accounts, IAM Identity Center provides centralized access management, allowing users to sign in once and access multiple accounts and applications without needing separate IAM users in each account.

Trust Policies vs Permission Policies

A role has two types of policies attached: a trust policy, which defines who (which principal) is allowed to assume the role, and one or more permission policies, which define what the role is allowed to do once assumed.

Policy Evaluation Logic

When a request is made, AWS evaluates all applicable policies (identity-based, resource-based, permission boundaries, and Service Control Policies at the organization level) together. The evaluation logic is: if any applicable policy contains an explicit Deny, the request is denied, full stop, regardless of any Allow elsewhere. If there is no explicit Deny, AWS checks whether any policy contains an explicit Allow; if so, the request is permitted. If neither an explicit Allow nor Deny is found, the default is implicit Deny. This means an explicit Deny always wins, which is a common source of confusion when a user has an "AdministratorAccess" policy attached but is still denied due to an unrelated explicit Deny statement elsewhere, such as a Service Control Policy.

Cross-Account Access

The `sts:AssumeRole` API operation allows a principal in one AWS account to temporarily assume a role defined in another account, provided the role's trust policy explicitly permits that account (or a specific principal within it) to assume it. This is the standard mechanism for cross-account access, and is preferred over creating separate IAM users with long-lived credentials duplicated across accounts.

Common Use Cases

- Granting an ECS task read-only access to a specific S3 bucket
- Allowing a CI/CD pipeline to push images to ECR without embedding static credentials
- Federating corporate directory users into AWS via SAML
- Cross-account access using role assumption (sts:AssumeRole)

Best Practices

- Never use the root account for daily operations; create an IAM user or role instead.
- Rotate access keys regularly, and prefer roles with temporary credentials over long-lived access keys wherever possible.
- Use policy conditions to restrict access by IP range, VPC endpoint, or time of day where appropriate.
- Regularly review IAM Access Analyzer findings and unused credentials reports.